

Strings and regular expressions

Recap: regular expressions

A *regular expression* is a pattern used to find matches in text.

Example: suppose I want to extract just the lecture number from the following file name. How would I do that?

```
1 "teaching/sta279-f23/slides/lecture_22.qmd"
```

pattern ; number after — (and before .)

— \d+ → _ 22

(?<_>\d+ → 22

Recap: regular expressions

A *regular expression* is a pattern used to find matches in text.

Example: suppose I want to extract just the lecture number from the following file name. How would I do that?

```
1 str_extract("teaching/sta279-f23/slides/lecture_22.qmd", "\\d+")
```

```
[1] "279"
```

```
1 str_extract("teaching/sta279-f23/slides/lecture_22.qmd", "_\\d+")
```

```
[1] "_22"
```

```
1 str_extract("teaching/sta279-f23/slides/lecture_22.qmd",  
2             "(?<=_)\\d+")
```

```
[1] "22"
```

Recap: regular expressions

Last time, we learned the following regular expression tools:

- `\d` matches any digit (in R, have to type `\\d` because we write the regex in a string)
- `.` matches any character (except `\n`)
- `+` means “at least once”
- `(?<=)` and `(?=)` are positive lookbehinds and lookaheads
- `|` is alternation (one pattern or another)

More regular expressions

```
1 strings <- c("apple", "banana", "raspberry",  
2             "blackberry", "grrreat", "random")
```

How would I select just raspberry and blackberry?

More regular expressions

```
1 strings <- c("apple", "banana", "raspberry",  
2             "blackberry", "grrreat", "random")
```

How would I select just raspberry and blackberry?

```
1 str_view(strings, "berry")
```

```
[3] | rasp<berry>  
[4] | black<berry>
```

More regular expressions

```
1 strings <- c("apple", "banana", "raspberry",  
2             "blackberry", "grrreat", "random")
```

How would I select “raspberry”, “blackberry”, “grrreat”, and “random”?

More regular expressions

```
1 strings <- c("apple", "banana", "raspberry",  
2             "blackberry", "grrreat", "random")
```

How would I select “raspberry”, “blackberry”, “grrreat”, and “random”?

```
1 str_view(strings, "r")
```

```
[3] | <r>aspbe<r><r>y  
[4] | blackbe<r><r>y  
[5] | g<r><r><r>eat  
[6] | <r>andom
```

More regular expressions

```
1 strings <- c("apple", "banana", "raspberry",  
2             "blackberry", "grrreat", "random")
```

How would I select just “raspberry”, “blackberry”, and “grrreat”?

More regular expressions

```
1 strings <- c("apple", "banana", "raspberry",  
2             "blackberry", "grrreat", "random")
```

How would I select just “raspberry”, “blackberry”, and “grrreat”?

```
1 str_view(strings, "rr+")
```

```
[3] | raspbe<rr>y
```

```
[4] | blackbe<rr>y
```

```
[5] | g<rrr>eat
```

happens twice



```
1 str_view(strings, "r{2}")
```

```
[3] | raspbe<rr>y
```

```
[4] | blackbe<rr>y
```

```
[5] | g<rr>reat
```

More regular expressions

```
1 strings <- c("apple", "banana", "raspberry",  
2             "blackberry", "grrreat", "random")
```

How would I select just “grrreat”?

```
1 str_view(strings, "r{3}")
```

```
[5] | g<rrr>eat
```

More regular expressions

```
1 strings <- c("apple", "banana", "raspberry",  
2             "blackberry", "grrreat", "random") `aubergine`
```

How would I select “apple”, “raspberry”, “blackberry”, and “grrreat”?

Option 1: check for 'e'

option 2: two repeated characters

More regular expressions

```
1 strings <- c("apple", "banana", "raspberry",  
2             "blackberry", "grrreat", "random")
```

How would I select “apple”, “raspberry”, “blackberry”, and “grrreat”?

```
1 str_view(strings, "(.)\\1")
```

[1]		a<pp>le			
[3]		raspbe<rr>y	capture	group	(using parentheses to capture a particular pattern)
[4]		blackbe<rr>y			
[5]		g<rr>reat			

↳ one occurrence of any character

\\1 : back reference
refers to a previously captured group
(what we captured, again)

(.)\\1 : any character, and then that same character again

More regular expressions

```
1 strings <- c("papa", "banana", "memento",  
2             "blackberry", "grrreat", "random")
```

How would I select “papa”, “banana”, and “memento”?

$(..)\|1$
↑ ↑
any two these same two characters again
characters

More regular expressions

```
1 strings <- c("papa", "banana", "memento",  
2             "blackberry", "grrreat", "random")
```

How would I select “papa”, “banana”, and “memento”?

```
1 str_view(strings, "(..)\1")
```

```
[1] | <papa>  
[2] | b<anan>a  
[3] | <meme>nto
```

```
1 str_view(strings, "(..)+")
```

```
[1] | <papa>  
[2] | <banana>  
[3] | <mement>o  
[4] | <blackberry>  
[5] | <grrrea>t  
[6] | <random>
```

More regular expressions

```
1 "The mean  $\mu$  is defined by  $\mu = \frac{1}{n} \sum_i x_i$ "
```

How would I extract μ and $\mu = \frac{1}{n} \sum_i x_i$?

pattern : starts with $\$, ends with $\$$$

maybe : $\underbrace{\$. + \$}_{\text{something}}$

More regular expressions

```
1 "The mean  $\mu$  is defined by  $\mu = \frac{1}{n} \sum_i x_i$ "
```

How would I extract μ and $\mu = \frac{1}{n} \sum_i x_i$?

```
1 str_extract("The mean  $\mu$  is defined by  $\mu = \frac{1}{n} \sum_i x_i$ "  
2               "\\$.+\\$")
```

```
[1] " $\mu$  is defined by  $\mu = \frac{1}{n} \sum_i x_i$ "
```

issue: regular expressions are greedy!
↳ by default, get biggest match

More regular expressions

```
1 "The mean  $\mu$  is defined by  $\mu = \frac{1}{n} \sum_i x_i$ "
```

How would I extract μ and $\mu = \frac{1}{n} \sum_i x_i$?

Option 1:

```
1 str_extract_all("The mean  $\mu$  is defined by  $\mu = \frac{1}{n} \sum_i x_i$ ",  
2 "\\$.+?\\$")
```

```
[[1]]
```

```
[1] "$\mu$"
```

```
x_i$"
```

? , means "don't be greedy"

```
"$\mu = \frac{1}{n} \sum_i
```

More regular expressions

```
1 "The mean  $\mu$  is defined by  $\mu = \frac{1}{n} \sum_i x_i$ "
```

How would I extract μ and $\mu = \frac{1}{n} \sum_i x_i$?

Option 2:

```
1 str_extract_all("The mean  $\mu$  is defined by  $\mu = \frac{1}{n} \sum_i x_i$ ",  
2 "\\$(^[\\$]+\\$")
```

```
[[1]]
```

```
[1] "$\\mu$"
```

```
x_i$"
```

```
"$\\mu = \\frac{1}{n} \\sum_i
```

[] : character class (a defined group of characters)

[^] : everything except specified characters

[^ \\\$] : every character except \$

Homework 5

- Strings and regular expressions for Great British Bake Off episodes
- Preview to functions and iteration!